

GitHub Repository: <https://github.com/krishsolanki1/COMP3011CW1>

API Documentation PDF: <https://github.com/krishsolanki1/COMP3011CW1/blob/main/docs/api.pdf>

Presentation Slides:

Dataset Used: <https://www.kaggle.com/datasets/ayeshaimran1619/bmw-sales-and-pricing-trends>

Introduction.

This coursework requires the design and implementation of a small, data-driven web service that exposes a non-trivial data model through a RESTful API. I chose to build a backend service for exploring used BMW prices using a real dataset sources through Kaggle. The end system allows for clients to store metadata about specific BMW models, attach market records (year and price), and query various analytics endpoints that summarise the dataset.

The service is implemented using FastAPI on top of a SQLite Database, with SQLAlchemy for object-relational mapping and Alembic for migrations. On top of basic CRUD operations, it provides endpoints for price-based analytics and protects write operations with a simple API key mechanism. The code is supported by a small pytest test suite and an import script that loads the Kaggle CSV data into the database. The remainder of this report describes the dataset, the architecture and API design, the analytics layer, and the main security, testing and design trade-offs, before briefly outlining limitations, future work, and how GenAI was used during development. The API is publicly deployed on PythonAnywhere at <https://krish1802.pythonanywhere.com>, and an MCP (Model Context Protocol) server is included that exposes the analytics endpoints as callable tools for AI assistants such as Claude.

Dataset Description.

The system is built around a BMW Used car listings dataset downloaded from Kaggle, “BMW Sales & Pricing Trends” (Author: Ayesha Imran). The original CSV contains one row per listing with attributes such as `model`, `year`, `price`, `mileage`, `fuelType`, `transmission`, and other basic features. For this project I concentrated on the fields that best supported simple market analysis: the car model name, year of registration, and listed price, with fuel type and transmission used as optional metadata on the model.

Each distinct model string in the CSV is treated as a separate `CarModel` entity in the database. For every row that has a valid model, year and price, the import script creates a corresponding `MarketRecord` linked to the correct `CarModel`. This produces a clear separation between model-level metadata (for example, whether the car is a diesel automatic) and record-level attributes (the year and advertised price of a specific listing). Rows that lack essential information are effectively ignored by the importer so that the analytics are only based on usable data.

The dataset is realistic but not perfect. It only covers BMWs listed on a particular platform and is unlikely to be representative of all regions or market conditions. Important factors such as mileage, optional extras and accident history are not modelled in depth, even though they clearly influence price. As a result the analytics should be interpreted as simple descriptive summaries rather than rigorous pricing models. However, the dataset is large enough and varied enough to support a genuinely non-trivial relational schema (one-to-many between models and records) and to motivate aggregate endpoints, which fits the goals of the coursework.

System Architecture & Design.

The BMW Market Analytics API is designed using a layered, modular architecture so that request handling, validation, business logic, security and persistence are clearly separated. The system is implemented in Python using FastAPI, SQLAlchemy and SQLite, with Alembic managing database migrations. This stack was chosen to balance rapid development, clarity of design and maintainability, while still being realistic enough to resemble a small production codebase.

FastAPI was selected as the web framework because it offers strong support for type-hinted request and response models via Pydantic, automatic input validation, and dependency injection for managing cross-cutting concerns such as database sessions and authentication. It also generates interactive Swagger documentation at `/docs`, which sped up testing and will be useful during the oral demonstration. Compared to more heavyweight frameworks such as Django, FastAPI has less boilerplate and a more API-focused design, which is appropriate for a coursework project that does not need a full templating or admin system.

Persistence is handled by SQLite, accessed through SQLAlchemy’s ORM. SQLite was chosen because it requires no external database server, runs as a simple file, and integrates cleanly with SQLAlchemy, which keeps setup friction low for local execution and the oral demo. Alembic migrations act as the single source of truth for the schema, so the database can be recreated consistently on a fresh clone rather than relying on manual table creation. To keep the relational model robust, `MarketRecord` rows reference `CarModel` via a foreign key with cascade delete behaviour, and SQLite foreign key enforcement is enabled so referential integrity is actually enforced at runtime. I also added lightweight integrity/performance improvements that directly support the analytics queries (e.g. year/price validation at the API layer and indexes aligned with common filter/group-by patterns such as `(car_model_id, year)`).

Figure 1 illustrates the high-level architecture of the system:

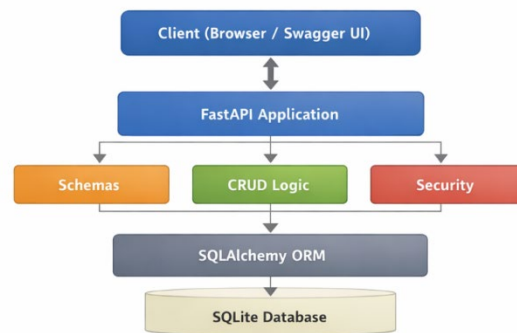


Figure 1: High-level architecture of the BMW Market Analytics API.

The client (for example, a browser using Swagger UI or a tool such as Postman) sends HTTP requests to the FastAPI application. The application routes each request to the correct endpoint function, which uses Pydantic schemas to validate input and structure output. A dedicated CRUD layer encapsulates data access logic, so routes do not need to deal with raw SQL or session management. Security dependencies enforce the API key check for protected endpoints. SQLAlchemy translates Python objects and queries into SQL operations against the SQLite database. This layered structure improves readability and makes it easier to evolve or test individual parts of the system.

The core data model consists of two relational entities: `CarModel` and `MarketRecord`. A `CarModel` row stores fields such as `id`, `name`, `series`, `body_style`, `fuel_type` and `transmission`. A `MarketRecord` row stores a foreign key back to `CarModel` along with year and price. Each `CarModel` can therefore have multiple associated `MarketRecord` entries, forming a one-to-many relationship.

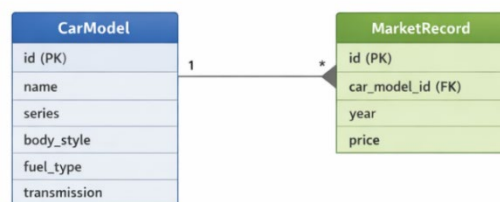


Figure 2: Entity-relationship diagram showing the one-to-many relationship between `CarModel` and `MarketRecord`.

In this schema, `MarketRecord.car_model_id` is a foreign key referencing `CarModel.id`. This relationship enables aggregation queries used in the analytics endpoints, such as calculating average price per model or generating yearly price trends. By normalising the schema into separate tables, the design avoids data duplication and ensures referential integrity.

In addition to the REST API, the project includes an MCP (Model Context Protocol) server in `mcp_server.py`. MCP is an open standard from Anthropic that allows AI assistants such as Claude to call external tools at runtime. The BMW MCP server exposes five tools: `list_models`, `get_model`, `get_average_price`, `get_price_trend`, and `get_top_models` each of which makes an HTTP request to the live PythonAnywhere deployment and returns a formatted result. The server uses the stdio transport from the Python mcp SDK and can be connected to Claude Desktop via a simple config entry. This extends the architecture beyond a traditional HTTP API: the same data and analytics layer becomes directly consumable by AI agents, demonstrating how a REST API can serve as a backend for both human clients and AI-driven workflows.

Overall, the architecture prioritises modularity and clarity. Route handlers focus on HTTP semantics, the CRUD layer centralises database logic, and configuration and security live in a separate core module. This reflects common patterns used in real FastAPI projects and demonstrates deliberate architectural decision-making rather than ad-hoc implementation.

API Design & Functionality.

The API is designed around a small set of coherent resources. The `/models` endpoints operate on `CarModel` entities and provide full CRUD functionality. Clients can list all models, create a new model, retrieve a model by ID, partially update selected fields via `PATCH`, or delete a model (which also removes its associated records). For each of these operations, Pydantic schemas define the structure of request and response bodies: dedicated “create” and “update” schemas control

which fields are required or optional on input, and “output” schemas limit what data is exposed to callers. This decouples the external contract from the internal ORM representation and makes the API easier to evolve.

Market price observations are exposed as a nested resource under a model via `/models/{id}/records`. This reflects the underlying one-to-many relationship: a record cannot exist without an associated model. Clients can list all records for a given model and add new ones. Nesting the routes in this way makes the API intuitive and avoids the need for clients to manipulate foreign keys directly.

The API uses standard HTTP methods and status codes: `GET` for retrieval, `POST` for creation, `PATCH` for partial update and `DELETE` for deletion. Successful reads produce `200 OK`, creations use `201 Created`, and deletions return `204 No Content` with an empty body. Attempts to access non-existent models or records result in `404 Not Found`, and write operations without a valid API key yield `401 Unauthorized`. These consistent responses make the API predictable and easier to consume. FastAPI’s automatic documentation support means that all endpoints appear in an interactive Swagger UI at `/docs`, which complements the separate written API documentation (provided as `docs/api.pdf`) by giving an executable view of the interface.

Analytics & Data Processing.

A key requirement for this coursework was to go beyond simple CRUD and show useful computation over the dataset. To address this, the project includes three analytics endpoints, each implemented as a read-only aggregate over the underlying tables.

The `/analytics/average-price` endpoint computes the overall average price for a single model across all its records. Internally it runs a query filtering `MarketRecord` by `car_model_id` and then applying `AVG(price)`. The response returns both the model’s ID and name alongside the calculated average. This endpoint is a simple but concrete example of how to derive summary statistics from the raw listings.

The `/analytics/price-trend` endpoint returns a price trend over time for a given model. It groups all records for that model by `year`, computing the average price and number of records in each group. The results are ordered by year so that clients can easily plot a line chart or otherwise reason about how the typical advertised price changes over time. While this does not model depreciation explicitly, it provides a useful proxy for how a given BMW model’s price behaves across different registration years.

The `/analytics/top-models` endpoint focuses on comparisons between models in a single year. It joins the `CarModel` and `MarketRecord` tables, filters records for a specific year, groups them by model, and then orders models by their average price in descending order. A `limit` query parameter controls how many models are returned. This exposes a simple ranking of “most expensive” models, along with how many listings each average is based on. Together, these analytics endpoints demonstrate aggregation, grouping and joining logic, and they show that the system does something meaningful with the imported dataset rather than merely storing it.

Security Considerations.

Within the scope of this coursework, I implemented a simple API key mechanism rather than a full authentication and authorisation system. A dependency function in the `core` module reads the `X-API-Key` header from each request and compares it with the `API_KEY` value loaded from the `.env` configuration. If the header is missing or the value does not match, the dependency raises an HTTP 401 error with a standard JSON error message. This dependency is attached to all write endpoints, including model creation, updates, deletions and record insertion.

Read-only endpoints, such as `GET /models` and the analytics routes, are left open intentionally. This reflects a common pattern where public data is readable by anyone, but modifications require a secret. It also simplifies demonstration and marking, because the markers can explore the analytics without worrying about credentials, while still seeing that the project addresses access control explicitly.

The API key approach is intentionally minimal and has clear limitations. It is essentially a shared secret: there are no individual user accounts, no role-based permissions, and no rate limiting. In a real deployment the service would also require HTTPS to protect the key in transit, and a more complete solution would likely involve a user database, token-based authentication (for example, JWT) and possibly an API gateway. For the purposes of this coursework, however, the API key mechanism provides a reasonable trade-off between complexity and demonstration value, and it shows that security considerations have been integrated into the design.

Testing Strategy.

Testing is implemented using `pytest` together with FastAPI’s `TestClient`. A fixture in `tests/conftest.py` constructs a reusable `TestClient` bound to `app.main:app`, which removes repeated setup boilerplate and keeps individual tests short and readable. The same fixture setup also centralises API key handling (via the `X-API-Key` header) so authenticated write endpoints can be exercised consistently across the suite.

The suite is organised by feature (health, models, records, analytics, auth) and focuses on realistic end-to-end behaviour over the real SQLite database seeded from the BMW CSV. Coverage includes: a liveness check for GET /health (200 OK and expected JSON); core read paths for models (GET /models returns a list, GET /models/{id} returns the correct resource when using an ID obtained from the list); and explicit negative cases to validate error handling (404 for non-existent IDs, and 401 for write endpoints without a valid API key). The tests also cover successful write flows: creating a model, creating a linked market record, and verifying the new data is visible via corresponding read endpoints, with clean-up at the end of each write test to avoid polluting the shared database state.

Beyond basic CRUD, analytics endpoints are exercised using real imported data: average-price is checked for an existing model with records (asserting a valid average_price field alongside model identity), and top-models is verified for response structure (year, results, average_price, num_records), using a year taken from existing records to avoid fragile hard-coding. Finally, two targeted “behaviour” tests improve confidence in correctness: a PATCH semantics test asserts that only provided fields change (others remain unchanged), and a validation test asserts that invalid record payloads (e.g. out-of-range year or non-positive price) are rejected with a 422 response.

Limitations & Future Work.

There are several known limitations in the current system. At the infrastructure level, the use of SQLite means the service is best suited to a single-user or small development environment, and it would not scale well under heavy concurrent load. The import script is a one-off batch process; it expects a local CSV file and does not offer any UI or scheduling. The list endpoints support pagination (skip/limit) and filtering by key fields (fuel type, transmission, price range, year), though more advanced filtering such as free-text search or multi-value filters is not currently implemented.

The dataset itself has limitations. It only covers BMWs advertised in a particular context and does not control for important factors such as mileage, previous ownership, or optional equipment. The analytics endpoints therefore provide simple descriptive statistics rather than rigorous market analysis, and results should be interpreted cautiously.

If I had more time, there are several directions I would explore. On the API side, further filtering options such as multi-value fuel type filters or free-text search across fields could be added to make the list endpoints more flexible. On the analytics side, I could extend the endpoints to show price distributions by fuel type or to segment models into “cheap”, “mid-range”, and “expensive” price bands per year. From a security perspective, replacing the API key with a proper user system and token-based auth would make the service more realistic. Finally, a simple front-end to visualise price trends would make the analytics easier to understand for non-technical users.

GenAI Usage Reflection.

Throughout this coursework I used two GenAI tools: ChatGPT (GPT-4) and Claude Code (claude-sonnet-4-6, Anthropic's CLI coding agent). The tools were used at different stages and at different levels of engagement.

In the earlier stages of the project, ChatGPT was used at a low-to-medium level: suggesting an initial modular project structure for FastAPI and SQLAlchemy, providing examples of Alembic configuration and dependency injection, generating boilerplate for CRUD functions and pytest test cases, and helping draft sections of the README. These interactions are documented in appendix items A1–A4. While useful for accelerating setup, this kind of use is largely templating — I ran the generated code locally, debugged errors, and adapted suggestions to fit the dataset and brief. The final architecture and schema decisions were my own.

The more significant GenAI usage came later, when Claude Code was used as an integrated development tool throughout the terminal. Rather than asking for boilerplate, I used it to understand unfamiliar technologies and explore design alternatives. For example, when deploying to PythonAnywhere I encountered an incompatibility between FastAPI's ASGI interface and PythonAnywhere's WSGI-only server. Claude Code was used to understand the async event loop model, diagnose why existing ASGI-to-WSGI adapters were failing under uWSGI, and iteratively design a working solution — first a lifespan shim, then ultimately a custom bridge using asyncio.run() per request. This required understanding the ASGI scope protocol at a technical level, not simply generating code. Relevant conversation logs are included in appendix items A5–A6.

The highest-level GenAI use was implementing the MCP (Model Context Protocol) server. MCP is an emerging open standard from Anthropic for exposing tools to AI assistants. Having no prior knowledge of the protocol, I used Claude Code to understand the specification, explore how the stdio transport works, and implement a working server that exposes five tools backed by the live PythonAnywhere deployment. This goes beyond debugging or boilerplate — it represents using GenAI to understand and apply a cutting-edge standard that extends the API beyond traditional HTTP clients to AI agent workflows.

In all cases I retained responsibility for design decisions, ran and tested all code on both the local environment and the live deployment, and ensured I could explain every component. GenAI accelerated understanding but did not replace it.

```
i'm starting comp3011 cw1 from scratch. i've got fastapi + sqlalchemy + alembic + sqlite. can you give me a clean modular structure (not enterprise) and explain what each part does briefly. i want app/main.py endpoint, routers, schemas, crud, db session etc.

Here's a clean, coursework-friendly structure that still looks "proper" (modular, testable, easy to scale) without turning into enterprise overkill.

Recommended structure

Code
On/
├── app/
│   ├── main.py           # FastAPI app endpoint (app.main:app)
│   ├── __init__.py
│   ├── router.py         # Central router that includes all route modules
│   ├── deps.py           # Shared dependencies (db session, auth, get-or-404 helpers)
│   └── routers/
│       ├── __init__.py
│       ├── health.py      # /health
│       ├── models.py      # /models (CRUD)
│       ├── records.py     # /models/{id}/records
│       └── analytics.py   # /analytics/*
├── config/
│   ├── __init__.py
│   ├── config.py         # Settings (.env) and constants (API_KEY, DATABASE_URL)
│   └── security.py       # API key verification dependency
├── db/
│   ├── __init__.py
│   ├── base.py           # SQLAlchemy Base
│   ├── models.py         # ORM models (CarModel, MarketRecord)
│   ├── session.py        # engine + SessionLocal + get_db_dependency
│   └── schemas/
│       ├── __init__.py
```

A 1: ChatGPT-assisted planning of the initial FastAPI project layout, proposing a clean modular structure (routers, schemas, CRUD, DB session/config) to keep the codebase organised and scalable while staying lightweight for coursework.

```
i've already got basic tests for /health and /models. help me extend to like ~10-13 tests total including 401 auth checks, 404 cases, successful create with api key, records creation, and analytics endpoint checks. keep it simple. pytest + testclient.

Thought for 15s >

Here's a simple, solid extension to ~13 tests using pytest + FastAPI TestClient (no mocking), covering 200/401/404, successful writes, records, and analytics.

Test plan (13 tests)
• Health (1): /health returns 200 + expected JSON
• Models (5): list models. get model by id, 404 on missing model, 401 on create w/out key, 201 on create with key (cleanup)
• Records (3): list records for a model (find one with records), create record w/key (cleanup), 404 when model missing
• Analytics (3): average price ok, average price 404, top-models ok (use real year from records)
• Auth (1): wrong API key rejected (401)

Python
# tests/conftest.py
import os

import pytest
from fastapi.testclient import TestClient

# helps if someone runs pytest without env vars set.
os.environ.setdefault("DATABASE_URL", "sqlite://./app.db")
os.environ.setdefault("API_KEY", "change-me")
```

A 2: ChatGPT-assisted expansion of the pytest suite beyond baseline checks, outlining a ~13-test plan that covers happy-path reads, common failure cases (401/404), authenticated writes (create model/record with cleanup), and core analytics endpoint validation using FastAPI's TestClient.

```
i want a one-command script to verify everything: install deps, migrations, seed/reset, run pytest. give me a windows powershell version and a bash version + what i need to add in my README

Thought for 2m 29s >

PowerShell
# scripts/verify.ps1
Set-ScriptParameters -ScriptName "Verify"
Set-ScriptMode -Version Latest

# run from repo root on relative paths (alembic.ini, .env) before
$repoRoot = Resolve-Path (Join-Path $PSScriptRoot ".")
Set-Location $repoRoot

# pick python (prefer python, fallback to py -3)
$pythonize = $null
if ($? {Get-Command python -ErrorAction SilentlyContinue}) {
    $pythonize = "python"
} elseif ($? {Get-Command py -ErrorAction SilentlyContinue}) {
    $pythonize = "py -3"
} else {
    throw "python not found. Install Python 3.11+ and ensure 'python' or 'py' is on PATH."
}

# Run the script
$pythonize .\scripts\verify.ps1
```

A 3: ChatGPT-assisted reproducibility improvement: generating a one-command verification script to automate dependency installation, Alembic migrations, database seeding/reset, and running the full pytest suite, with the corresponding README commands for quick end-to-end validation.

```
analytics.py
python

does this analytics code query the db in a sensible way? i'm using sqlite + sqlalchemy. can you point out any obvious performance issues and quick fixes (like indexes or query tweaks)?

Thought for 22s >

Yeah — for SQLite + SQLAlchemy this is sensible, straightforward query code you're doing single aggregate queries (AVG, COUNT, GROUP BY) with tight filters, and you're not doing anything accidentally No 1-nish inside Python loops.

A few obvious performance wins (and they're all "small change, big payoff"):

1) Add the right indexes (this is the main one)
Your hot paths are:
• average-price: SELECT market_records.car_model_id ...
  Add an index on market_records.car_model_id (or ensure the FK column is indexed).
• price_trend: SELECT car_model_id ... GROUP BY year ORDER BY year
  Best index composite ((car_model_id, year)).
• top-models: SELECT year ... GROUP BY car_model_id ORDER BY avg(price)
  Index on (year, car_model_id) and ideally composite starting with year: ((year, car_model_id)) (SQLite likes the leading column matching your filter).

If you already added year + ((car_model_id, year)) you're basically covered, adding (year, car_model_id) is the extra "top-models specifically" boost.
```

A 4: ChatGPT-assisted review of the analytics query layer, identifying low-effort performance improvements (indexes on car_model_id, year, and composite keys) and a small query tidy-up to better align SQLite execution plans with the API's common filter/group-by patterns

```
WSGI itself works fine — the issue is entirely with a2wsgi. Let me bypass it completely with a manual bridge that uses asyncio.run() per request (no external dependency needed).

In the Bash console:

nano ~/COMP3011CW1/wsgi.py

Replace with:

import asyncio
import io

from app.main import app as fastapi_app

__STATUS__ = {
    200: "OK", 201: "Created", 204: "No Content",
    400: "Bad Request", 401: "Unauthorized", 404: "Not Found",
    405: "Method Not Allowed", 422: "Unprocessable Entity",
    500: "Internal Server Error",
}

# ... (rest of the code) ...
```

A6: Claude Code-assisted diagnosis and resolution of PythonAnywhere WSGI/ASGI incompatibility — identifying that a2wsgi was failing under uWSGI and designing a custom asyncio.run() bridge to manually construct the ASGI scope per request, eliminating the external dependency entirely.

```
That version of a2wsgi doesn't support the lifespan kwarg. Fix it with a manual lifespan shim instead.

In the PythonAnywhere Bash console, run:

nano ~/COMP3011CW1/wsgi.py

Replace the entire content with:

from a2wsgi import ASGIMiddleware
from app.main import app as fastapi_app

async def app(scope, receive, send):
    if scope["type"] == "lifespan":
        return
    await fastapi_app(scope, receive, send)

application = ASGIMiddleware(_app)

Save (Ctrl+O), Enter, (Ctrl+X), then click Reload in the Web tab.
```

A5: Claude Code-assisted diagnosis of PythonAnywhere deployment failure — identifying that the installed version of a2wsgi did not support the lifespan parameter and designing a manual ASGI lifespan shim as an interim fix, demonstrating iterative debugging of an ASGI/WSGI compatibility issue in a live deployment environment.